EXERIMENT 14

Question: Write the Python program to implement Alpha-Beta pruning algorithm for gaming.

Aim:

To write and implement a Python program using the Alpha-Beta pruning algorithm to optimize the Minimax algorithm in gaming.

Algorithm:

Start the program.

Initialize the Tic-Tac-Toe game board.

Display the board and accept the human player's move.

Check whether the move is valid and update the board.

Check whether a player has won or the game is a draw.

Apply the Alpha-Beta pruning algorithm to evaluate possible computer moves.

Update the alpha and beta values to eliminate branches that cannot affect the final decision.

Select and execute the best move for the computer.

Repeat until a player wins or the game ends in a draw.

Display the final result and stop the progra

Code:

lab14.py - C:/Users/ELCOT/AppData/Local/Programs/Python/Python314/lab14.py (3.14.6)

File Edit Format Run Options Window Help

```
import math

# Count how many leaf nodes we actually visit (to show pruning benefit)
nodes_visited = 0

def alpha_beta(depth, node_index, is_max, values, alpha, beta, max_depth):
    global nodes_visited

    # Base case: reached a leaf node
    if depth == max_depth:
        nodes_visited += 1
        return values[node_index]

    if is_max:
        best = -math.inf
        # Two children for every node (binary tree)
        for i in range(2):
            value = alpha_beta(depth + 1, node_index * 2 + i,
                               False, values, alpha, beta, max_depth)
            best = max(best, value)
            alpha = max(alpha, best)

            # Pruning condition
            if beta <= alpha:
                print(f"Pruned at depth {depth} (MAX node), "
                      f"alpha={alpha}, beta={beta}")
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            value = alpha_beta(depth + 1, node_index * 2 + i,
                               True, values, alpha, beta, max_depth)
            best = min(best, value)
            beta = min(beta, best)

            # Pruning condition
            if beta <= alpha:
                print(f"Pruned at depth {depth} (MIN node), "
                      f"alpha={alpha}, beta={beta}")
                break
```

Output:

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/ELCOT/AppData/Local/Programs/Python/Python314/lab14.py ===
Pruned at depth 2 (MAX node), alpha=6, beta=5
Pruned at depth 1 (MIN node), alpha=5, beta=2

Leaf values        : [3, 5, 6, 9, 1, 2, 0, -1]
Best value for MAX : 5
Leaf nodes visited : 5 out of 8
>>>
```

Result:

Thus, the Python program implementing the Alpha-Beta pruning algorithm for gaming was executed successfully, selecting suitable moves while reducing unnecessary search.